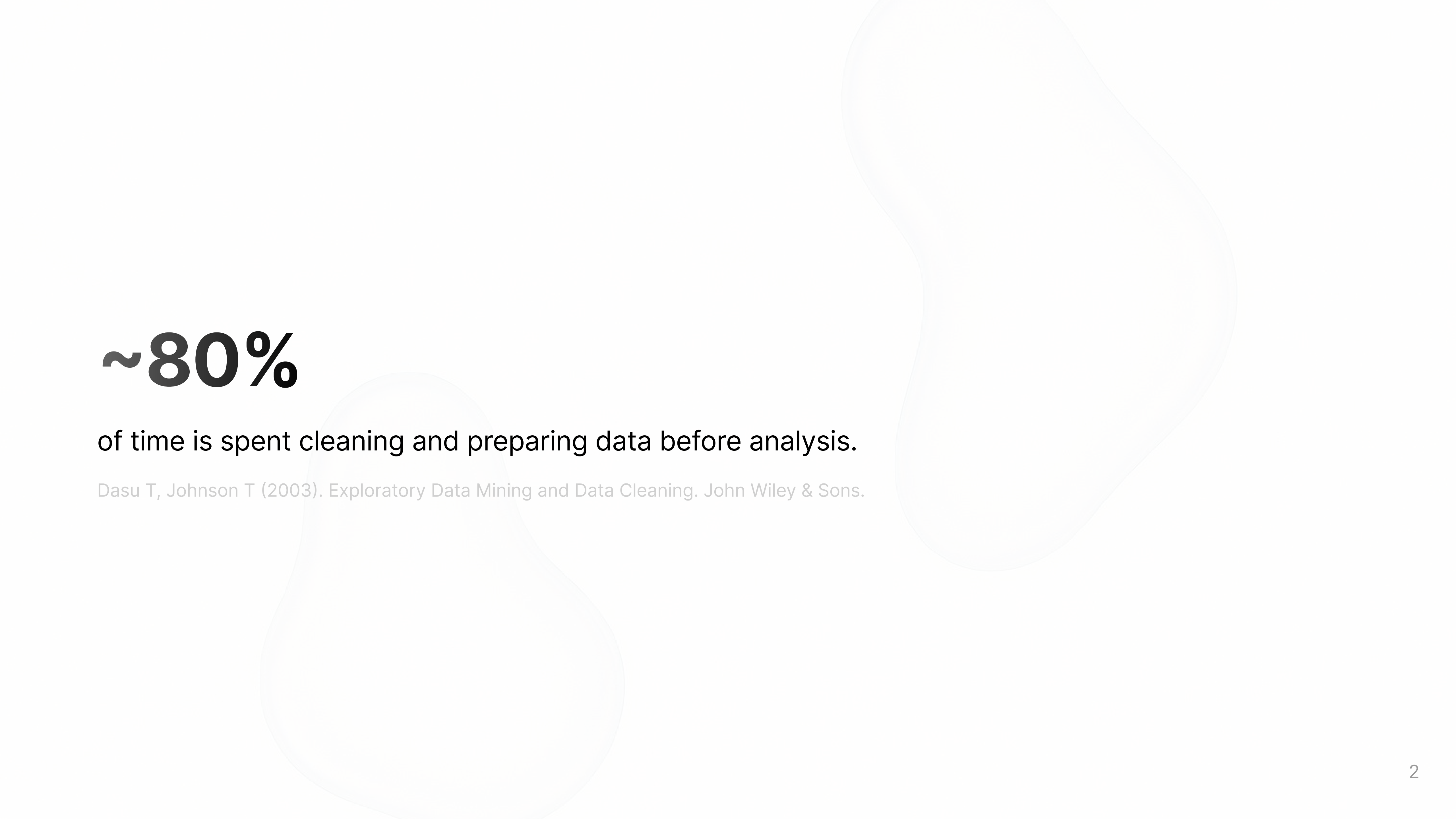


Data Wrangling & Tidy Data



~80%

of time is spent cleaning and preparing data before analysis.

Dasu T, Johnson T (2003). Exploratory Data Mining and Data Cleaning. John Wiley & Sons.

Agenda



Definitions

Tidy Data, Data Wrangling, Tools



Steps of data wrangling

Reshaping, Combine, Handling Missing Data, Transformation and Normalization



Practice tutorial

Recognize patterns and apply the concepts

Tidy Data

is a standard way of structuring datasets to facilitate manipulation, visualization and modeling

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280420583

each **variable** is a **column**;
each **column** is a **variable**

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280420583

each **observation** is a **row**;
each **row** is an **observation**.

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280420583

each **value** is a **cell**;
each **cell** is a single **value**

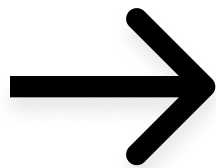
Messy vs Tidy

Column headers are values, not variable names

[2]

religion	<\$10k	\$10–20k	\$20–30k	\$30–40k	\$40–50k	\$50–75k
Agnostic	27	34	60	81	76	137
Atheist	12	27	37	52	35	70
Buddhist	27	21	30	34	33	58
Catholic	418	617	732	670	638	1116
Don't know/refused	15	14	15	11	10	35
Evangelical Prot	575	869	1064	982	881	1486
Hindu	1	9	7	9	11	34
Historically Black Prot	228	244	236	238	197	223
Jehovah's Witness	20	27	24	24	21	30
Jewish	19	19	25	25	30	95

Messy



[2]

religion	income	freq
Agnostic	<\$10k	27
Agnostic	\$10–20k	34
Agnostic	\$20–30k	60
Agnostic	\$30–40k	81
Agnostic	\$40–50k	76
Agnostic	\$50–75k	137
Agnostic	\$75–100k	122
Agnostic	\$100–150k	109
Agnostic	>150k	84
Agnostic	Don't know/refused	96

Tidy

5

Messy vs Tidy

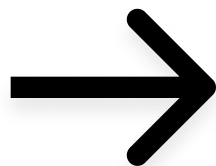
Multiple variables stored in one column

[2]

[2]

country	year	column	cases
AD	2000	m014	0
AD	2000	m1524	0
AD	2000	m2534	1
AD	2000	m3544	0
AD	2000	m4554	0
AD	2000	m5564	0
AD	2000	m65	0
AE	2000	m014	2
AE	2000	m1524	4
AE	2000	m2534	4
AE	2000	m3544	6
AE	2000	m4554	5
AE	2000	m5564	12
AE	2000	m65	10
AE	2000	f014	3

Messy



country	year	sex	age	cases
AD	2000	m	0–14	0
AD	2000	m	15–24	0
AD	2000	m	25–34	1
AD	2000	m	35–44	0
AD	2000	m	45–54	0
AD	2000	m	55–64	0
AD	2000	m	65+	0
AE	2000	m	0–14	2
AE	2000	m	15–24	4
AE	2000	m	25–34	4
AE	2000	m	35–44	6
AE	2000	m	45–54	5
AE	2000	m	55–64	12
AE	2000	m	65+	10
AE	2000	f	0-14	3

Tidy

6

Data Wrangling

transforms and maps data through a series of steps
to become clean, consistent, reliable and useful for its intended application.

~ibm

Goal: prepare data for analysis by converting it into a tidy format.



Tools

for data wrangling

pandas



melt(), pivot(), merge(),
groupby(), agg()

tidyverse



dplyr, tidyr

Data Wrangler



AI-Assisted data wrangling
extension

Import & Export Data

in pandas

Import



```
pd.read_csv(),  
pd.read_excel(),  
pd.read_json()
```

Inspect



```
df.head(),  
df.info(),  
df.describe()
```

Export



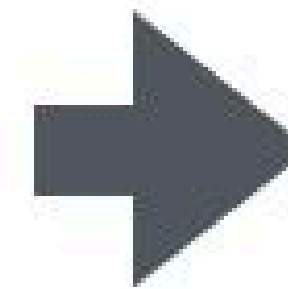
```
df.to_csv(),  
df.to_excel(),  
df.to_json()
```

Reshape Data

melt

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



df3.melt(id_vars=['first', 'last'])

[3]

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

- wide to long
- column headers into row values

Reshape Data

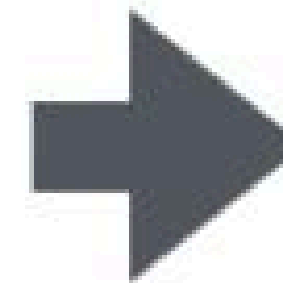
pivot

[3]

df

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

Stacked



bar	A	B	C
foo			
one	1	2	3
two	4	5	6

Record

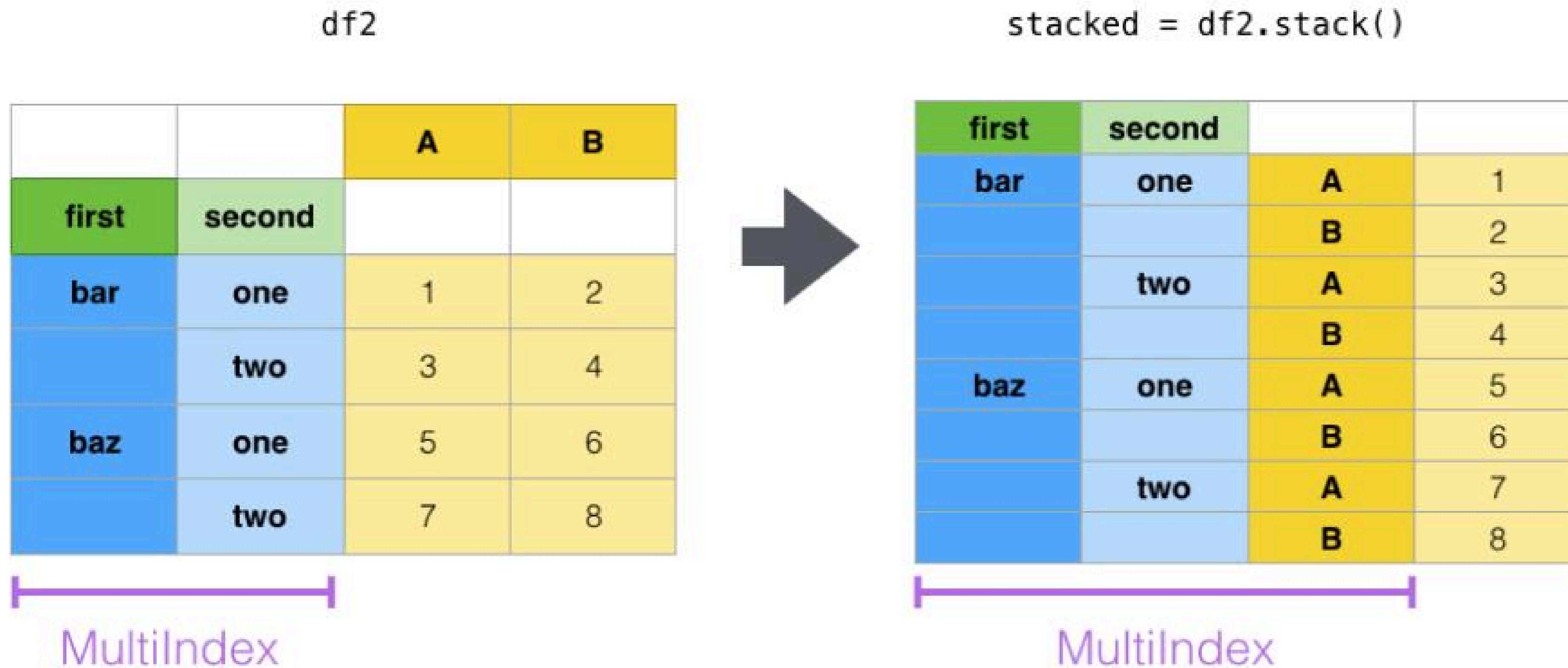
```
df.pivot(index='foo',  
          columns='bar',  
          values='baz')
```

- long to wide
- sort out data where a single observation is scattered over multiple rows

Reshape Data

stack

[3]



similar to melt but with multi level indices

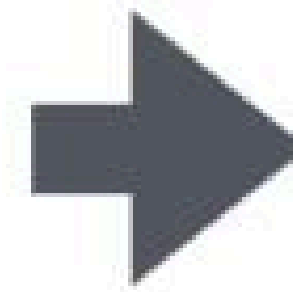
Reshape Data

unstack

stacked

first	second		
bar	one	A	1
		B	2
	two	A	3
		B	4
baz	one	A	5
		B	6
	two	A	7
		B	8

MultiIndex



stacked.unstack()

		A	B
first	second		
bar	one	1	2
	two	3	4
baz	one	5	6
	two	7	8

MultiIndex

[3]

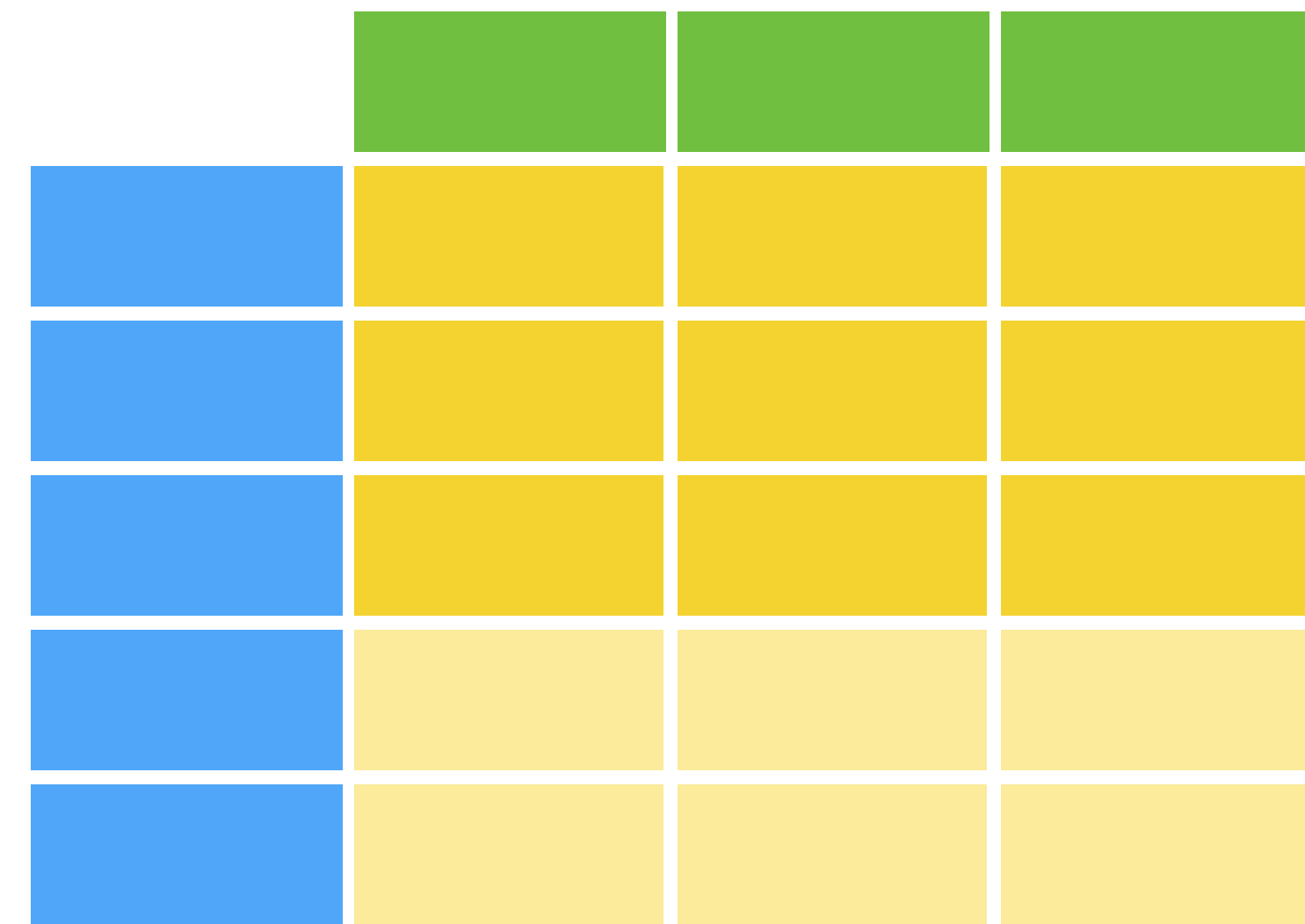
similar to pivot but with multi level indices

Combine Data

concat



`pd.concat()`

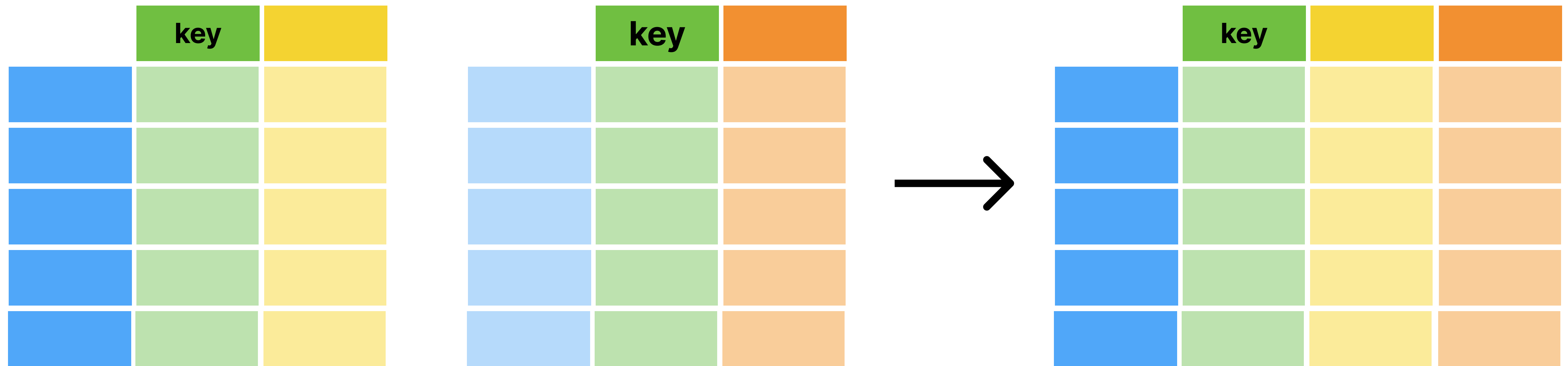


- append similar rows (observations)
- two or more data frames with the same index or the same columns

Combine Data

merge

```
pd.merge(left, right,  
on=["key"], how="inner")
```



- join two datasets based on a key column
- when different tables have related data
- left / right / inner / outer

Handling Missing Data

Kind of Absence

Structural

Data that cannot logically exist
(e.g. “number of pregnant males”)

Observational

Data that should exist but
wasn't recorded

How it Appears

Explicit

Visibly missing in the dataset
→ presence of absence

Implicit

Hidden due to dataset's structure
(e.g. category combination not present)
→ absence of presence

Explicit Missing

Common types:

- np.nan for numeric columns
- pd.NA for all nullable dtypes
- None for Object dtypes

	A	B	C	D
0	1.0	2.0	apple	10.0
1	NaN	3.0	None	20.0
2	-99.0	NaN	banana	NaN
3	4.0	5.0	mango	40.0
4	5.0	6.0	pear	50.0

Fill / Impute:

```
df.dropna(axis=0) ...  
df.dropna(axis=1) ...  
df.dropna(how="all") ...  
df.dropna(thresh=3) ...  
df.interpolate() ...
```



	A	B	C	D
0	1.0	2.0	apple	10.0
1	0.0	3.0	0	20.0
2	-99.0	0.0	banana	0.0
3	4.0	5.0	mango	40.0
4	5.0	6.0	pear	50.0

Remove:

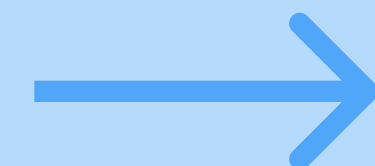
```
df.dropna(axis=0) ...  
df.dropna(axis=1) ...  
df.dropna(how="all") ...  
df.dropna(thresh=3) ...
```



	A	B	C	D
0	1.0	2.0	apple	10.0
3	4.0	5.0	mango	40.0
4	5.0	6.0	pear	50.0

Replace coded missing values:

```
df.replace({-99: pd.NA})
```



	A	B	C	D
0	1.0	2.0	apple	10.0
1	NaN	3.0	None	20.0
2	<NA>	NaN	banana	NaN
3	4.0	5.0	mango	40.0
4	5.0	6.0	pear	50.0

Implicit Missing

	year	qtr	price
0	2020	1	1.88
1	2020	2	0.59
2	2020	3	0.35
3	2020	4	NaN
4	2021	2	0.92
5	2021	3	0.17
6	2021	4	2.66

Two missing observations:

Explicitly missing → price for Q4 2020

Implicitly missing → Q1 2021



qtr	1	2	3	4
year				
2020	1.88	0.59	0.35	NaN
2021	NaN	0.92	0.17	2.66

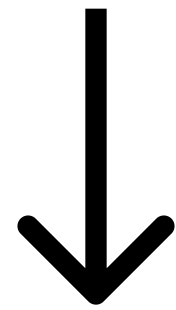
```
df.pivot(columns="qtr",  
         values="price",  
         index="year")
```

Make them explicit

Use pivoting, reshaping or re-indexing to reveal gaps

Missing Values in Categorical Variables

	name	smoker	age
0	Ikaia	no	34
1	Oletta	no	88
2	Leriah	previously	75
3	Dashay	no	47
4	Tresaun	yes	56



	name	smoker	age
0	Ikaia	no	34
1	Oletta	no	88
2	Leriah	previously	75
3	Dashay	no	47

value_counts()

```
smoker
no          3
previously  1
yes         0
Name: count, dtype: int64
```

groupby()

```
smoker
no          56.333333
previously  75.000000
yes          NaN
Name: age, dtype: float64
```



**Can be used to
remove categories:**

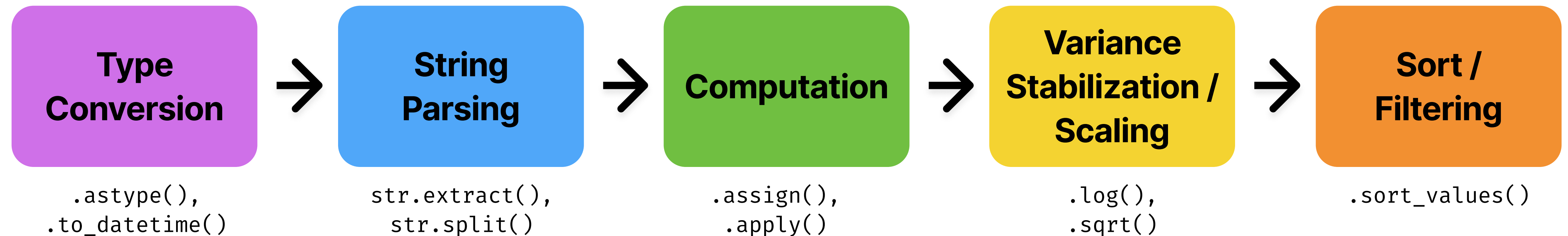
```
df_cut["smoker"].cat
.remove_unused_categories()

smoker
no          3
previously  1
Name: count, dtype: int64
```

Transformation

“Changing data values to make them suitable for analysis”

Modifying or creating new variables to enhance or prepare data for modeling.



Case Study: Transforming a Weather Dataset

	id	year	month	element	d1	d2	d3
0	MX17004	2010	1	tmax	27.0	24.0	None
1	MX17004	2010	1	tmin	14.0	10.0	None
2	MX17004	2010	1	tmax	NaN	NaN	None

Messy (Wide) Data

	id	year	month	element	day	value
0	MX17004	2010	1	tmax	1.0	27.0
1	MX17004	2010	1	tmin	1.0	14.0
3	MX17004	2010	1	tmax	2.0	24.0
4	MX17004	2010	1	tmin	2.0	10.0

Tidy (Long) Data



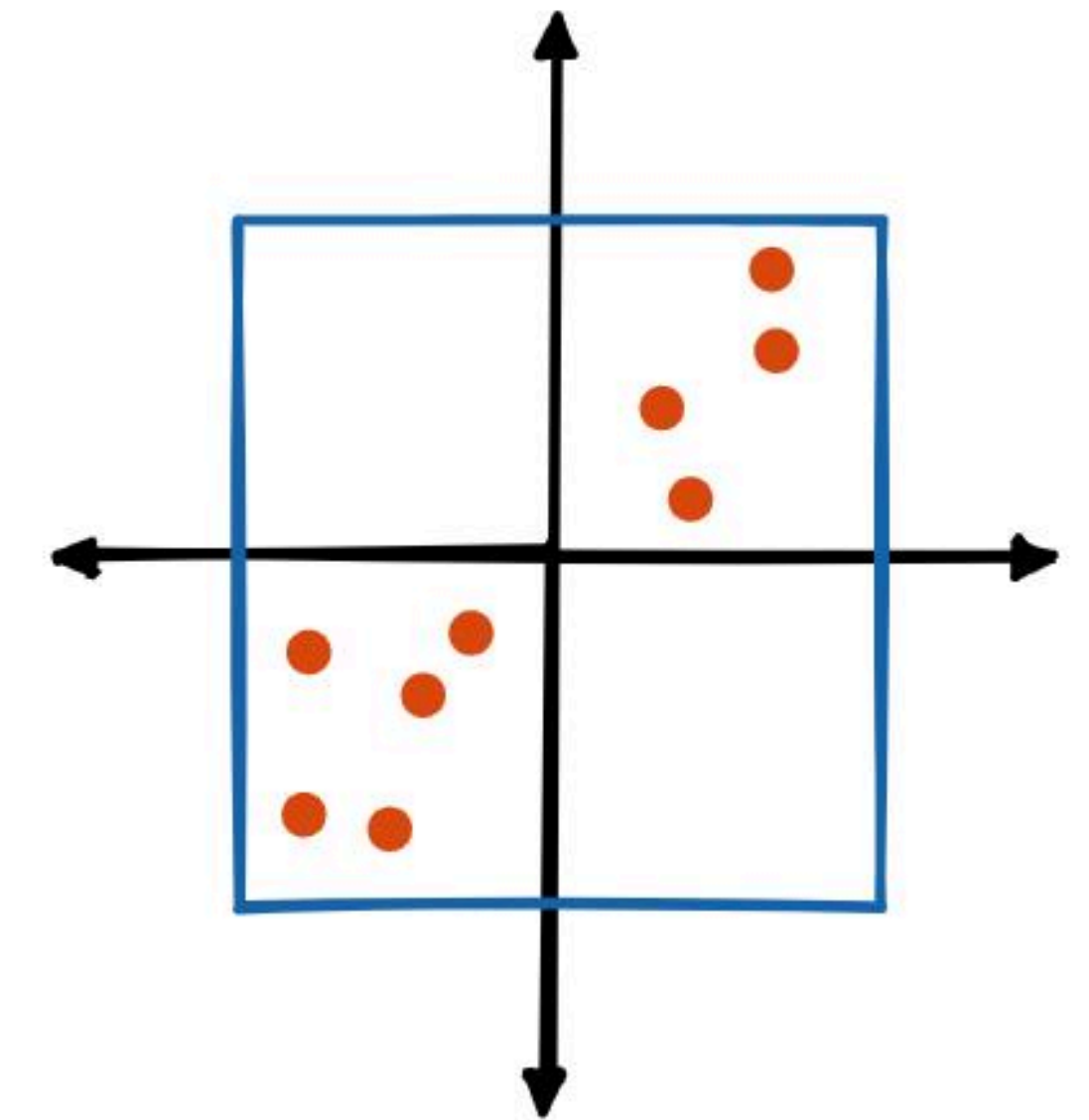
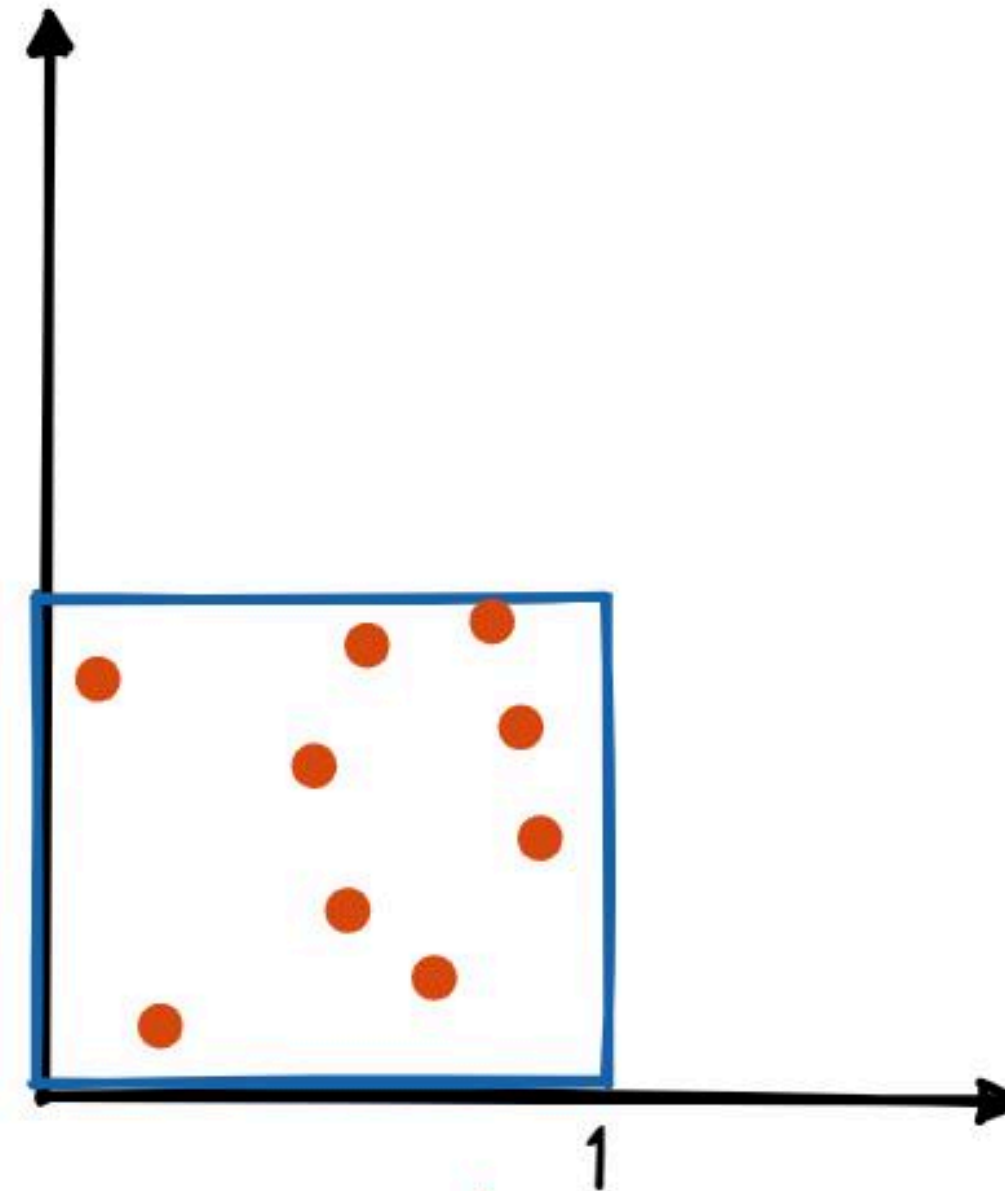
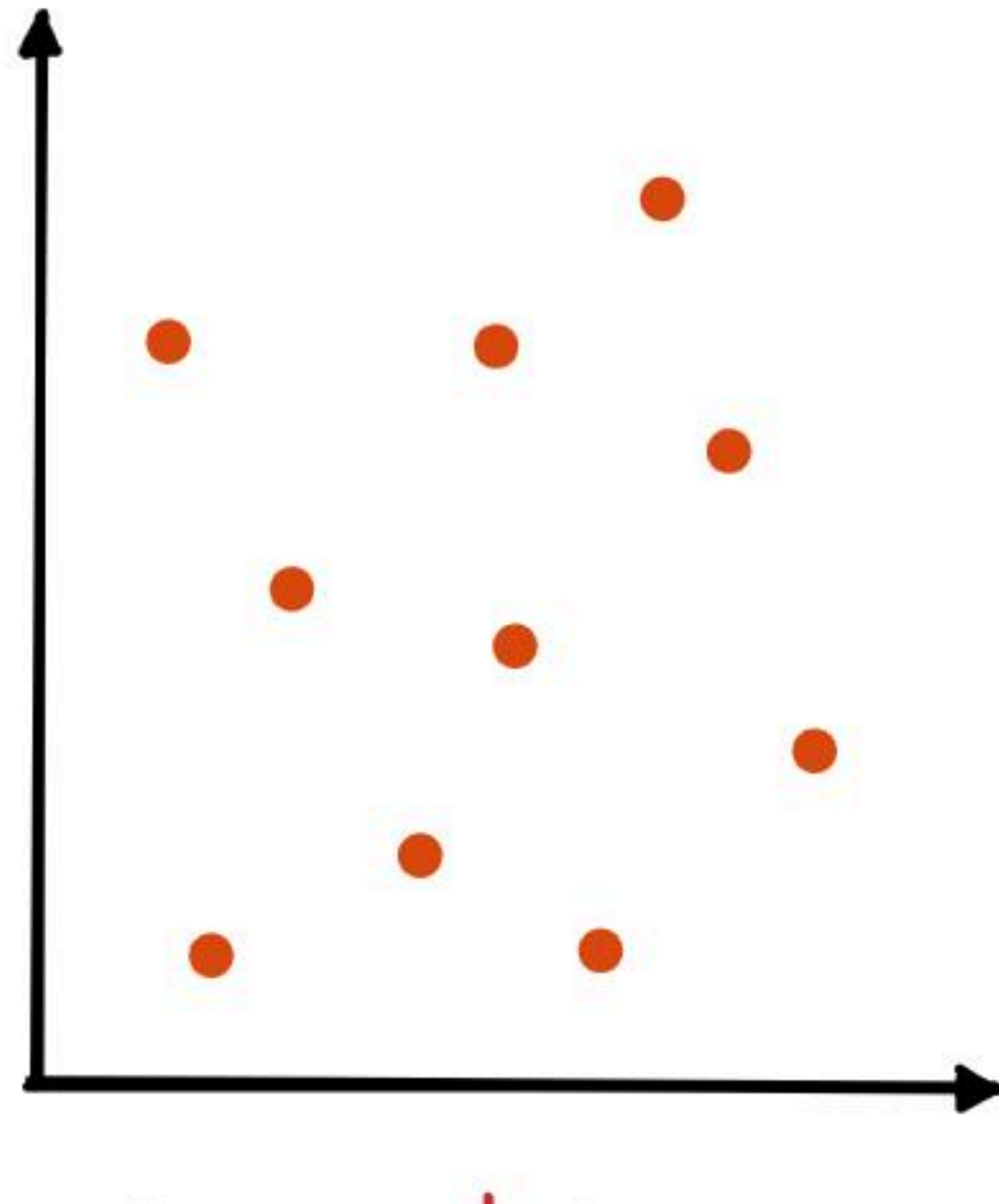
```
tidy_weather = (  
    ... weather  
    ... .melt(  
        ... id_vars=["id", "year", "month", "element"],  
        ... var_name="day",  
        ... value_name="value",  
    ... )  
  
    )  
tidy_weather.loc[:, "day"] = (  
    ... tidy_weather["day"]  
    ... .str.extract("(\d+)")  
    ... .astype(float)  
    )  
tidy_weather = (  
    ... tidy_weather  
    ... .dropna(subset=["value"])  
    ... .sort_values(["id", "year", "month", "day"])  
    )
```

Transformation Steps

Normalization

scaling numeric values into a common range

[4]



required for a lot of tasks
e.g machine learning

Min-Max Normalization
Keeps relative differences
(0-1)

Z-Score Standardization
Centers data at 0 with unit variance
(e.g. -3 to +3)

Extra: AI Assisted Data Wrangling

Automating tedious transformations with AI

Data Wrangler Extension

Uses ML

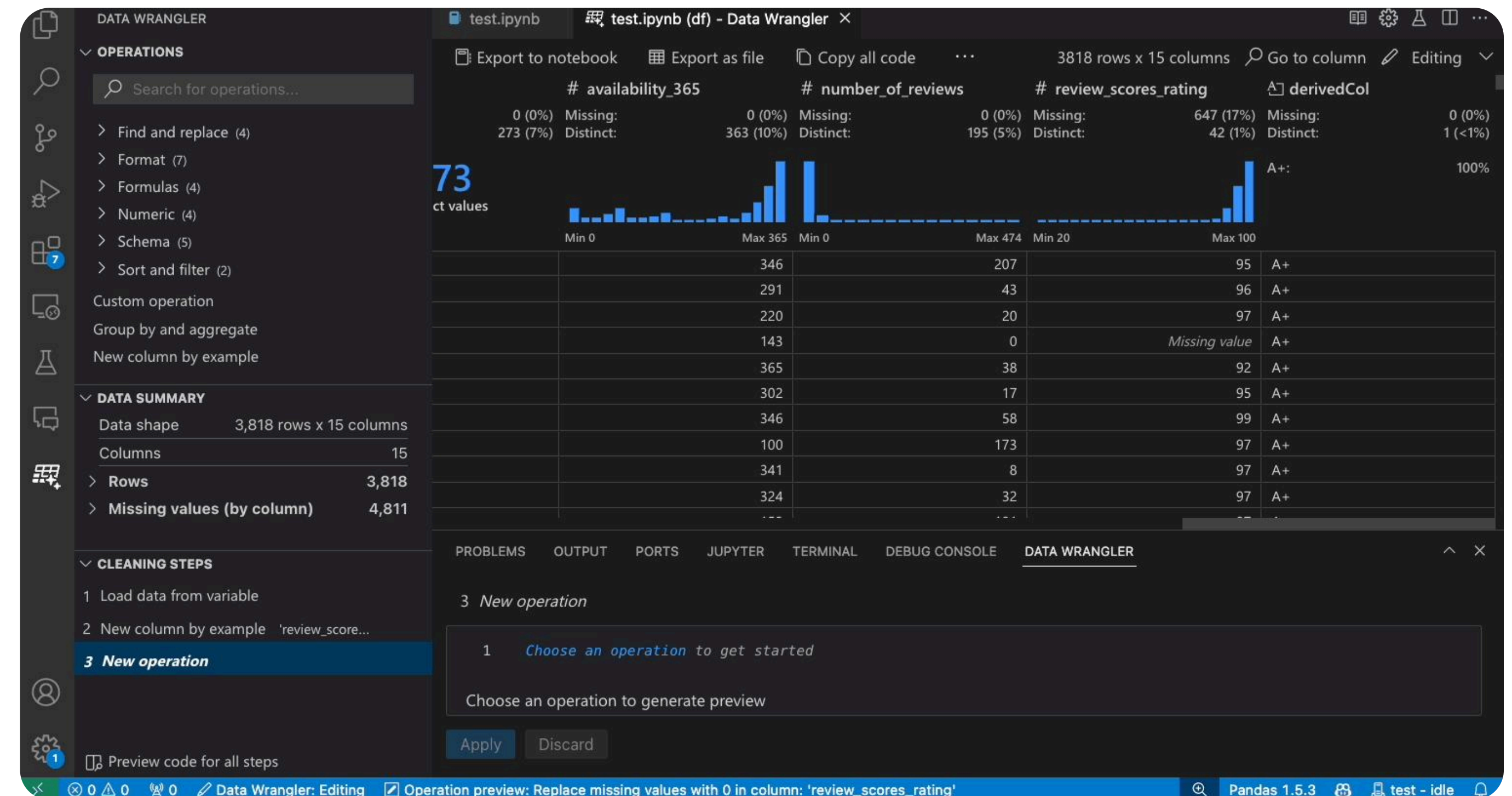
To automatically suggest Pandas transformations

Supports multiple operations

Like type conversion, column renaming, filtering, grouping, NA handling

Generates Code

Clean Python / Pandas code for instant reproducibility



DATA WRANGLER

OPERATIONS

Search for operations...

> Find and replace (4)

> Format (7)

> Formulas (4)

> Numeric (4)

> Schema (5)

> Sort and filter (2)

Custom operation

Group by and aggregate

New column by example

DATA SUMMARY

Data shape

3,818 rows x 15 columns

Columns

15

Rows

3,818

Missing values (by column)

4,811

CLEANING STEPS

Load data from variable

New column by example 'review_score...

New operation

Preview code for all steps

test.ipynb

test.ipynb (df) - Data Wrangler

Export to notebook

Export as file

Copy all code

...

3818 rows x 15 columns

Go to column

Editing

availability_365

number_of_reviews

review_scores_rating

derivedCol

0 (0%) Missing: 273 (7%)

0 (0%) Missing: 363 (10%)

0 (0%) Missing: 195 (5%)

647 (17%) Missing: 42 (1%)

0 (0%) Missing: 1 (<1%)

73

ct values

Min 0

Max 365

Min 0

Max 474

Min 20

Max 100

	346	207	95	A+
	291	43	96	A+
	220	20	97	A+
	143	0	Missing value	A+
	365	38	92	A+
	302	17	95	A+
	346	58	99	A+
	100	173	97	A+
	341	8	97	A+
	324	32	97	A+
	...	...	...	...

PROBLEMS

OUTPUT

PORTS

JUPYTER

TERMINAL

DEBUG CONSOLE

DATA WRANGLER

3 New operation

1 Choose an operation to get started

Choose an operation to generate preview

Apply

Discard

0

0

Data Wrangler: Editing

Operation preview: Replace missing values with 0 in column: 'review_scores_rating'

Pandas 1.5.3

test - idle

24

Summary

Data wrangling turns raw information into **tidy data** for structured insights

Sources

<https://pandas.pydata.org/docs>

<https://aeturrell.github.io/python4DS>

<https://www.ibm.com/think/topics/data-wrangling>

https://www.researchgate.net/publication/215990669_Tidy_data

Image Sources

- [1] <https://d33wubrfki0l68.cloudfront.net/6f1ddb544fc5c69a2478e444ab8112fb0eea23f8/91adc/images/tidy-1.png>
- [2] https://www.researchgate.net/publication/215990669_Tidy_data
- [3] <https://aeturrell.github.io/python4DS/data-tidy.html>
- [4] <https://marketplace.visualstudio.com/items?itemName=ms-toolsai.datawrangler>
- [5] https://storage.googleapis.com/algodailyrandomassets/curriculum/standvsnorm/comparison_graph.png

Tutorial



<https://github.com/justingeber/datawrangling-tidydata>

Thank you

